

ATmega32A Module

UART (Serial Communication)

Ali Sahafi

1 Theory

UART (*Universal Asynchronous Receiver and Transmitter*) sends data one bit after the other. It uses one wire to send (TXD) and one wire to receive (RXD), so it can send and receive at the same time (full duplex). There is *no clock wire*. For this reason, both sides must use the same settings before they start:

- the same **speed** (the *baud rate*, in bits per second), and
- the same **frame**: how many data bits, parity or no parity, and how many stop bits.

The frame. When nothing is sent, the line is HIGH (idle). Every byte is sent as one frame:

Start bit	Data bits	Parity bit	Stop bit(s)
always 0 (LOW)	8 bits, LSB first	optional	always 1 (HIGH), 1 or 2 bits

The most common setting is **8N1**: 8 data bits, No parity, 1 stop bit. So one byte needs 10 bits on the wire.

Baud rate. At 9600 baud, one bit takes $1/9600 \approx 104 \mu\text{s}$. The AVR makes the baud rate from the CPU clock with the register UBRR:

$$\text{UBRR} = \frac{F_{\text{CPU}}}{16 \cdot \text{Baud}} - 1 \qquad \text{Baud} = \frac{F_{\text{CPU}}}{16 \cdot (\text{UBRR} + 1)}$$

UBRR must be a whole number, so the real baud rate is a little different from the one you want. The error should be smaller than about 2 %. At 8 MHz:

Wanted baud	UBRR	Real baud	Error
4800	103	4808	0.2 %
9600	51	9615	0.2 % (we use this)
19200	25	19231	0.2 %
38400	12	38462	0.2 %
57600	16 (double speed)	58824	2.1 % (too high)
115200	8 (double speed)	111111	-3.5 % (too high, does not work well)

ASCII. UART sends bytes (numbers 0–255). To send *text*, every character has a number: the ASCII code. For example 'A' = 65, 'a' = 97, '0' = 48, '1' = 49, new line '\n' = 10, carriage return '\r' = 13. A terminal program on the PC shows each received byte as the character with that code. So if you send the *byte* 65, the terminal shows A. If you want the terminal to show the *number* 65, you must send the two characters '6' and '5'.

2 How to use the driver

The driver gives you one object: `UART`. You start it with one function call:

```
UART.begin(9600); // 9600 baud, 8N1
UART.begin(9600, UART_PARITY_EVEN, 2); // 9600 baud, 8 data bits, even parity, 2 stop bits
```

The options

Argument	Value	Meaning
baudRate	9600, 19200, ...	Speed in bits per second. Write the number you want. The driver calculates <code>UBRR</code> for you.
parity	UART_PARITY_NONE (default) UART_PARITY_EVEN UART_PARITY_ODD	No parity bit Even parity: the number of 1s (data + parity) is even Odd parity: the number of 1s (data + parity) is odd
stopBits	1 (default), 2	Number of stop bits

The driver always uses 8 data bits. The other side (for example the terminal program on the PC) must use the *same* settings.

Functions

Function	What it does
UART.begin(baud, parity, stopBits)	Set up and start the UART (see the options above)
UART.write(byte)	Send one byte exactly as it is. <code>UART.write(65)</code> sends the byte 65, and the terminal shows A
UART.print("text")	Send text
UART.print('A')	Send one character
UART.print(number)	Send a number <i>as text</i> : <code>UART.print(65)</code> sends the two characters 6 and 5. Works for all integer types and for <code>float</code> (2 decimals; <code>UART.print(x, 4)</code> gives 4)
UART.println(...)	Same as <code>print()</code> , then a new line. <code>UART.println()</code> sends only the new line
UART.available()	Returns <code>true</code> if a received byte is waiting. It does not wait
UART.read()	Waits until a byte has arrived, then returns it
UART.onReceive(function)	Call <code>function(byte)</code> for every received byte (interrupt, see the interrupt lecture)

write() or print()? Use `write()` to send raw data (one byte, any value 0–255). Use `print()` and `println()` to send text that a person reads in the terminal.

Waiting or not waiting. `UART.read()` *waits* until a byte arrives: your program stops on this line until the user types something. If the program must do other work at the same time, first ask `UART.available()`, and only call `read()` when it returns `true`:

```
if (UART.available()) { // something has arrived?
    uint8_t data = UART.read(); // then read it (no waiting)
    ...
}
```

Interrupts (we come back to this in the interrupt lecture). `onReceive()` turns on the receive interrupt and calls `sei()` for you. Your function then runs automatically for every received byte. Keep this function short. If it changes a variable that `main()` also uses, declare the variable as `volatile`. To turn the interrupt off again, pass `nullptr`.

Driver functions and registers

In the lecture slides we set the registers by hand. Each driver function does these steps for you:

Driver function	Registers
UART.begin(9600)	UBRRH = 0; UBRL = 51; (UBRR = 8 MHz / (16 · 9600) - 1) UCSRC = (1<<URSEL) (1<<UCSZ1) (1<<UCSZ0); (8N1) UCSRB = (1<<RXEN) (1<<TXEN);
UART.begin(9600, UART_PARITY_EVEN, 2)	in UCSRC also: UPM1 = 1 (even parity), USBS = 1 (2 stop bits)
UART.write(x)	while (!(UCSRA & (1<<UDRE))); UDR = x;
UART.available()	UCSRA & (1<<RXC)
UART.read()	while (!(UCSRA & (1<<RXC))); return UDR;
UART.onReceive(f)	UCSRB = (1<<RXCIE); sei();

Double speed. If the normal UBRR gives an error of more than 1 %, the driver tries the double-speed mode (U2X in UCSRA, $UBRR = F_{CPU}/(8 \cdot \text{Baud}) - 1$) and uses it if the error is smaller. At 8 MHz this only happens for 57600 and 115200 baud.

3 Example: echo with ASCII code

Board hardware. The UART pins PD0 (RXD) and PD1 (TXD) are connected to the **FT232** chip on the board. This chip is a USB-to-UART converter. When you connect the board's USB cable to the PC, the PC gets a new serial port (a *virtual COM port*):

- **Windows:** COM3, COM4, ... (see *Device Manager* → *Ports*).
- **macOS:** /dev/cu.usbserial-...
- **Linux:** /dev/ttyUSB0

You also need the LEDs (D0–D7 on PORTB, active-low: LOW = ON) for Task 1.

Serial terminal. To see the data on the PC, you need a terminal program, for example the *Serial Monitor* extension in VS Code, CoolTerm or PuTTY. Choose the serial port of the board and these settings: **9600 baud, 8 data bits, no parity, 1 stop bit (8N1)**, no flow control.

Build and upload. `make uart`. Then open the terminal and type a character, for example A. The board answers `You typed: A (ASCII 65)`.

```
/*
 * UART Example -- echo every character together with its ASCII code
 * Build & flash: make uart
 *
 * Open a serial terminal on the PC: 9600 baud, 8 data bits, no parity,
 * 1 stop bit (8N1). Type a character, for example A, and the board answers:
 *   You typed: A (ASCII 65)
 */

#define F_CPU 8000000UL
#include "uart.hpp"

int main() {
    UART.begin(9600); // 8N1

    UART.println("ATmega32A ready. Type something!");

    while (true) {
        if (UART.available()) { // a byte has arrived (RXC flag)
            uint8_t c = UART.read(); // read it from UDR

            UART.print("You typed: ");
            UART.write(c); // the byte itself -> shown as a character
            UART.print(" (ASCII ");
            UART.print(c); // the same byte as a number -> "65"
            UART.println(")");
        }
    }
}
```

4 Laboratory Tasks

For each task, make a new .cpp file in the main project folder (for example Task1.cpp). Start the file with `#define F_CPU 8000000UL`. Then include the drivers you need: `drivers/uart/uart.hpp`, `drivers/gpio/gpio.hpp`, `drivers/timer/timer.hpp`. Upload it with `make SRC=Task1.cpp`. Use 9600 baud (8N1) in all tasks.

Task 1: Send and Receive Bytes (20 min)

- **Task 1.1 (Send):** Send the numbers 0 to 255 to the PC, one number every second. Use `UART.write(number)` and `_delay_ms(1000)`. Look at the data in the terminal. What do you see? Then switch the terminal to show the received bytes in HEX (for example in CoolTerm). Now what do you see? (The ASCII part of the lecture explains the difference.)
- **Task 1.2 (Receive):** Wait for a byte from the PC with `UART.read()` and show it on the 8 LEDs on PORTB. The LEDs are active-low, so write the inverted value: `GPIO.write(PORTB, ~data)`. Type the character A in the terminal. Which LEDs are on? Compare with the ASCII code of A in binary. Try a and 1 too.

Task 2: Send Your Name (ASCII Text)

Send your name to the PC, one time every second. Do it in two steps:

1. First send only one character every second, for example `UART.print('A')`.
2. Then send your whole name and a new line with `UART.println("Your Name")`.

`println()` adds the characters `'\r'` and `'\n'`, so every name starts on a new line in the terminal.

Assignment: Send and Receive at the Same Time

Write one program that does two things at the same time:

- It sends the numbers 0 to 255 to the PC as text with `UART.println(number)`, one number every second.
- It shows every byte that it receives from the PC on the LEDs on PORTB (active-low).

Rules:

- Use `Timer0` *without interrupts* for the 1 second, for example with a 10ms tick in `TIMER_CTC` mode and `Timer0.compareMatched()` (see the timer handout). Count 100 ticks for one second. Do *not* use `_delay_ms()`.
- Check for received data all the time with `UART.available()`. Do *not* call `UART.read()` without asking `UART.available()` first, or the program stops and waits.

The main loop then looks like this:

```
while (true) {
    if (Timer0.compareMatched()) { // 10 ms have passed
        ...                       // count ticks; after 100 ticks send the next number
    }
    if (UART.available()) {        // a byte has arrived
        ...                       // read it and show it on the LEDs
    }
}
```